

ЧАСТНА ПРОФЕСИОНАЛНА ГИМНАЗИЯ ЗА ДИГИТАЛНИ НАУКИ
„СОФТУНИ БУДИТЕЛ”, гр. София

ДЪРЖАВЕН ИЗПИТ ЗА ПРИДОБИВАНЕ НА ТРЕТА СТЕПЕН
НА ПРОФЕСИОНАЛНА КВАЛИФИКАЦИЯ

ДИПЛОМЕН ПРОЕКТ

на тема:

„Проследяване на физическа активност и тренировки”

Уеб приложение MaxFitness

професия- код: **481030, "Приложен програмист"**

специалност- код: **4810301, "Приложно програмиране"**

Изготвил: **Калоян-Максимилиан Йорданов Митов**

Клас: **12А**

Ръководител-консултант: **Мариян Йорданов**

Сесия: май-юни 2026г.

Дата:

София, 2026 г.

Съдържание

Глава 1: Увод и актуалност

Глава 2: Анализ на съществуващи решения

Глава 3: Изисквания към системата

Глава 4: Проектиране на архитектурата

Глава 5: Избор на технологии

Глава 6: Проектиране на базата данни

Глава 7: Реализация на сървърната част

Глава 8: Реализация на клиентската част

Глава 9: Тестване

Глава 10: Деплойване и заключение

Глава 1. Увод и актуалност

1.1. Въведение в проблематиката

В съвременното общество физическата активност и поддържането на здравословен начин на живот придобиват все по-голямо значение. Нарастващата седентарност, породена от дигитализацията на работните процеси и ежедневието, поставя сериозни предизвикателства пред общественото здраве в глобален мащаб. Световната здравна организация (СЗО) определя физическата неактивност като четвъртия по значимост рисков фактор за глобална смъртност, отговорен за приблизително 3,2 милиона смъртни случая годишно.

Според данни на СЗО от 2022 г., повече от 1,4 милиарда възрастни в световен мащаб са физически неактивни, което представлява около 27,5% от световното население. В България тази тенденция е особено изразена — национални проучвания показват, че значителна част от трудоспособното население не покрива минималните препоръчителни норми за седмична физическа активност. Тези данни подчертават необходимостта от систематичен подход към проследяване и стимулиране на физическата активност чрез съвременни технологични решения.

Дигиталните инструменти за проследяване на фитнес активността представляват обещаващ механизъм за промяна на поведенческите модели. Изследванията в областта на поведенческата психология демонстрират, че визуализацията на прогреса, поставянето на цели и социалната отчетност са ключови фактори за дългосрочното поддържане на физическата активност. Именно тези принципи стоят в основата на разработката на системата MaxFitness.

1.2. Цели и задачи на проекта

Основната цел на настоящия дипломен проект е проектирането, разработката и тестването на пълнофункционална уеб базирана система за управление на фитнес тренировки — MaxFitness. Системата е предназначена да предостави на потребителите интуитивен и ефективен инструмент за документиране на тренировъчната си активност, проследяване на прогреса и участие в мотивиращи общностни предизвикателства.

За постигане на тази основна цел са поставени следните конкретни задачи: анализ на съществуващите решения на пазара и идентифициране на техните предимства и

недостатъци; формулиране на функционални и нефункционални изисквания към системата въз основа на изследване на потребителските нужди; проектиране на архитектурата и базата данни на системата; реализация на сървърната и клиентската части на приложението; написване на автоматизирани тестове за верификация на функционалността; и настройка на CI/CD тръбопровод за автоматизирано изграждане и деплоиване.

Проектът се реализира с образователна цел в контекста на дипломна работа, като същевременно поставя за цел да демонстрира практическото приложение на съвременни технологии и архитектурни подходи в разработката на реална, полезна уеб система. Разработеното решение трябва да отговаря на стандартите за качество, сигурност и производителност, характерни за производствени системи.

1.3. Обхват на системата

Системата MaxFitness обхваща следните основни функционални области: регистрация и автентикация на потребители с ролево базиран контрол на достъпа; дневник на тренировките с възможност за добавяне, редактиране и изтриване на тренировъчни сесии; каталог от упражнения с описания и интерактивна SVG карта на тялото за визуализация на засегнатите мускулни групи; система от предизвикателства за мотивиране на потребителите; класация (leaderboard) за насърчаване на здравословна конкуренция; статистики и аналитика на тренировъчния прогрес; и административен панел за управление на съдържанието и потребителите.

Технологичният стек на системата включва ASP.NET Core 8 MVC като основна сървърна рамка, Microsoft SQL Server като система за управление на бази данни, Entity Framework Core с подход Code First за работа с данните, Razor Views за генериране на клиентската страна, Bootstrap 5 за адаптивен дизайн, ASP.NET Core Identity за управление на потребители и роли, GitHub Actions за CI/CD автоматизация, и Docker за контейнеризация на приложението.

1.4. Ограничения и допускания

В рамките на настоящия дипломен проект са приети определени ограничения, произтичащи от характера на проекта, наличните ресурси и времевия хоризонт. Системата е разработена като уеб приложение и не включва мобилно приложение, въпреки че интерфейсът е проектиран да бъде напълно адаптивен и използваем на

мобилни устройства. Интеграцията с носими устройства (wearables) и фитнес тракери не е реализирана в настоящата версия.

Системата не включва функционалност за хранителен дневник и проследяване на калориен прием, тъй като това би разширило значително обхвата на проекта извън рамките на дипломната работа. Прогнозирането на резултати и препоръките въз основа на изкуствен интелект не са обхванати в настоящата разработка, въпреки че представляват интересна посока за бъдещо развитие. Плащанията и абонаментните модели са извън обхвата на системата.

Относно допусканията: приема се, че потребителите имат достъп до интернет и използват съвременен уеб браузър. Системата е проектирана за използване предимно от индивидуални потребители, а не за управление на фитнес зали или организации. Данните, въвеждани от потребителите, се приема, че са коректни по отношение на физиологичните параметри, тъй като системата не разполага с механизми за верификация на реалността на въведените тренировъчни данни.

Структурата на настоящата дипломна работа следва логическата последователност на процеса на разработка на софтуер. В следващите глави са разгледани последователно: анализът на съществуващите решения (Глава 2), изискванията към системата (Глава 3), архитектурното проектиране (Глава 4), изборът на технологии (Глава 5), проектирането на базата данни (Глава 6), реализацията на сървърната част (Глава 7), реализацията на клиентската част (Глава 8), тестването (Глава 9), и деплойването и заключението (Глава 10).

Глава 2: Анализ на съществуващи решения

2.1 Strava — функции, силни и слаби страни

Strava е една от най-широко използваните платформи за проследяване на физическата активност в света, с над 100 милиона регистрирани потребители към 2024 година. Приложението е стартирало през 2009 година с фокус върху колоезденето и бягането, но с времето е разширило своите функционалности. Основната технология на Strava се базира на GPS проследяване, което позволява на потребителите да записват своите маршрути, скорост и изминато разстояние в реално време.

Сред основните силни страни на Strava се откроява изключително развитата социална компонента. Потребителите могат да следват профилите на приятели и спортисти, да оставят коментари и реакции, както и да участват в предизвикателства и клубове. Тази геймификация се е доказала като ефективен мотивационен инструмент. Интеграцията с голям брой носими устройства прави Strava централен хъб за спортните данни.

Въпреки предимствата си, Strava има съществени слабости. Значителна част от ключовите функции са заключени зад платения абонамент. Приложението е ориентирано предимно към издръжливостни спортове на открито и не предоставя структурирани планове за силови тренировки. Липсва и функционалност за проследяване на хранителния режим.

2.2 MyFitnessPal — хранене и калории

MyFitnessPal е безспорен лидер в сегмента на приложенията за проследяване на хранителния режим и калориен прием. Платформата разполага с база данни от над 14 милиона хранителни продукта. Потребителите могат да регистрират своите хранения чрез търсене или сканиране на баркод. Приложението автоматично изчислява дневния калориен прием и разпределението на макроеlementи.

Независимо от предимствата си в областта на храненето, MyFitnessPal демонстрира съществени ограничения в тренировъчната компонента. Приложението предлага само базово регистриране на физическа активност без структурирани тренировъчни програми. Социалната функционалност е ограничена. Значителна критика е отправена и към точността на хранителните данни.

2.3 Nike Training Club — тренировъчни програми

Nike Training Club представлява водещо приложение за структурирани тренировъчни програми. За разлика от Strava, NTC се фокусира върху предоставянето на готови тренировки с видео инструкции и аудио насоки. Приложението предлага над 185 безплатни тренировки с различна продължителност и ниво на трудност — силови, йога, кардио.

Ограниченията на NTC произтичат от липсата на функционалности извън тренировъчното ръководство. Приложението не предлага проследяване на хранителния режим, мониторинг на прогреса при вдигане на тежести или GPS проследяване. Прогресът се измерва единствено чрез брой завършени тренировки.

2.4 Garmin Connect и Fitbit

Garmin Connect е платформата за анализ на данни от Garmin устройства. Тя предоставя детайлни данни за сърдечен ритъм, кислородна сатурация и вариабилност на сърдечния ритъм. Fitbit, придобит от Google, е насочен към по-широка аудитория с проследяване на стъпки, сън и предизвикателства. И двете платформи са затворени екосистеми, зависещи от хардуерни устройства.

2.5 Сравнителен анализ и идентифициране на пропуски

Прегледът разкрива ясна фрагментация на пазара: всяка платформа е силна в конкретна ниша, но не предлага цялостно решение. Strava доминира в социалното проследяване на открито, MyFitnessPal в храненето, NTC в тренировъчното ръководство, а Garmin и Fitbit в здравния мониторинг. Тази фрагментация принуждава потребителите да използват множество приложения.

Анализът идентифицира няколко пропуски: липса на достъпна уеб платформа, обединяваща управлението на тренировки и мониторинг на прогреса; липса на персонализирано ръководство за начинаещи; липса на интегрирано управление на тренировъчното натоварване. Образователното съдържание и инструментите за периодизация също са рядкост.

2.6 Позициониране на разработваното приложение

На базата на идентифицираните пропуски, MaxFitness е проектирано като цялостна уеб-базирана платформа за широк кръг потребители. За разлика от Strava, MaxFitness е ориентиран към фитнес тренировки в зала и домашни условия. В сравнение с

MyFitnessPal, MaxFitness интегрира тренировъчната функционалност като ядро на системата. За разлика от Garmin и Fitbit, MaxFitness е напълно независимо от хардуерни устройства, достъпно за всеки с интернет.

Глава 3. Изисквания към системата

3.1. Методология за събиране на изисквания

Процесът на събиране и формулиране на изискванията към системата MaxFitness е проведен чрез комбинация от методи: анализ на конкурентните решения (разгледан в предходната глава), проучване на потребителски нужди чрез анкетиране на целевата група, и преглед на академичната литература в областта на проектирането на фитнес информационни системи. Изискванията са структурирани съгласно стандарта IEEE 830 за спецификация на софтуерни изисквания, разделени на функционални и нефункционални категории.

3.2. Функционални изисквания

Функционалните изисквания описват конкретното поведение на системата — какво трябва да прави тя. Те са организирани в следните функционални области: управление на потребители, управление на тренировки, управление на упражнения, предизвикателства и класация, статистики и отчети, и административни функции.

ФИ-01: Системата трябва да позволява регистрация на нови потребители с имейл адрес и парола. ФИ-02: Системата трябва да поддържа автентикация на регистрирани потребители. ФИ-03: Системата трябва да поддържа ролева система с минимум две роли — обикновен потребител (User) и администратор (Admin). ФИ-04: Системата трябва да позволява промяна на профилните данни, включително снимка на профила. ФИ-05: Системата трябва да позволява смяна на паролата. ФИ-06: Потребителите трябва да могат да изтрият своя акаунт.

ФИ-07: Системата трябва да позволява на потребителите да добавят тренировъчни сесии с дата, продължителност, тип тренировка и бележки. ФИ-08: Системата трябва да позволява добавянето на упражнения към тренировъчна сесия с брой серии, повторения и тегло. ФИ-09: Потребителите трябва да могат да редактират и изтриват своите тренировъчни сесии. ФИ-10: Системата трябва да предоставя хронологичен дневник на всички тренировки на потребителя. ФИ-11: Системата трябва да поддържа пагинация на дневника с тренировки.

ФИ-12: Системата трябва да поддържа каталог от упражнения с описания и категории. ФИ-13: Всяко упражнение трябва да бъде асоциирано с една или повече мускулни групи. ФИ-14: Системата трябва да предоставя интерактивна SVG карта на тялото за

визуализация на мускулните групи. ФИ-15: Администраторите трябва да могат да добавят, редактират и изтриват упражнения от каталога.

ФИ-16: Системата трябва да предоставя система от предизвикателства с ясно дефинирани цели и срокове. ФИ-17: Потребителите трябва да могат да се записват за активни предизвикателства. ФИ-18: Системата трябва да проследява напредъка на потребителите в записаните предизвикателства. ФИ-19: Системата трябва да поддържа публична класация (leaderboard) по различни критерии. ФИ-20: Класацията трябва да показва минимум топ 10 потребители по брой тренировки за текущата седмица/месец.

ФИ-21: Системата трябва да предоставя персонализирани статистики за потребителя — брой тренировки, общо времетраене, динамика по периоди. ФИ-22: Системата трябва да визуализира прогреса на потребителя чрез графики. ФИ-23: Администраторският панел трябва да предоставя обобщена информация за системата — брой потребители, тренировки, активни предизвикателства. ФИ-24: Администраторите трябва да могат да управляват потребителски акаунти.

3.3. Потребителски истории (User Stories)

ПИ-01: Като нов посетител на сайта, искам да мога да се регистрирам с имейл и парола, за да получа достъп до функционалностите на системата. Критерии за приемане: формулярът за регистрация изисква имейл, парола (мин. 6 символа) и потвърждение на паролата; при успешна регистрация потребителят е автоматично влязъл в профила си; при невалидни данни се показват конкретни съобщения за грешка.

ПИ-02: Като регистриран потребител, искам да мога да добавя нова тренировка с упражненията, изпълнени в нея, за да имам дигитален дневник на тренировъчната си активност. Критерии за приемане: формулярът позволява добавяне на дата, продължителност и тип тренировка; потребителят може да добавя произволен брой упражнения с серии, повторения и тегло; тренировката се запазва и появява в хронологичния дневник.

ПИ-03: Като редовен потребител на системата, искам да виждам историята на своите тренировки в хронологичен ред, за да мога да проследявам своята последователност и прогрес. Критерии за приемане: дневникът показва тренировките в обратен хронологичен ред; всяка тренировка показва датата, типа, продължителността и броя упражнения; дневникът поддържа пагинация при повече от 10 тренировки.

ПИИ-04: Като потребител, искам да разгледам каталога с упражнения и да видя коя мускулна група работи всяко упражнение, за да планирам по-добре тренировките си. Критерии за приемане: каталогът показва всички налични упражнения с описания; интерактивната SVG карта на тялото позволява кликуване върху мускулна група и показване на съответните упражнения; упражненията са категоризирани по тип (силови, кардио, гъвкавост).

ПИИ-05: Като мотивиран потребител, искам да се запиша за активно предизвикателство и да проследявам напредъка си към целта, за да остана мотивиран. Критерии за приемане: страницата с предизвикателства показва всички активни предизвикателства с цел, срок и брой участници; потребителят може да се запише с едно кликуване; напредъкът се актуализира автоматично при добавяне на нови тренировки.

ПИИ-06: Като конкурентен потребител, искам да виждам класацията с топ потребители по брой тренировки, за да знам своето място сред общността. Критерии за приемане: класацията показва топ 10 потребители за текущата седмица и месец; всеки потребител вижда своето място в класацията дори ако не е в топ 10; класацията се актуализира в реално време при добавяне на нови тренировки.

ПИИ-07: Като потребител, искам да виждам персонализирани статистики за своята тренировъчна активност — общо тренировки, общо часове, динамика — за да имам ясна представа за своя прогрес. Критерии за приемане: страницата со статистики показва общ брой тренировки, общо времетраене, средна продължителност; налична е графика на активността по месеци за последните 6 месеца; показват се най-популярните упражнения на потребителя.

ПИИ-08: Като потребител, искам да мога да редактирам и изтривам свои тренировки, за да коригирам грешно въведени данни. Критерии за приемане: всяка тренировка има бутони за редактиране и изтриване; само собственикът на тренировката може да я редактира или изтрива; при изтриване се иска потвърждение; след успешна операция потребителят получава уведомление.

ПИИ-09: Като администратор, искам да имам достъп до административен панел с обобщена информация за системата, за да мога да управлявам съдържанието и потребителите. Критерии за приемане: административният панел е достъпен само за потребители с роля Admin; панелът показва статистики за системата; администраторът може да управлява потребители, упражнения и предизвикателства.

ПИ-10: Като администратор, искам да мога да добавям нови упражнения към каталога с описания и асоциирани мускулни групи, за да поддържам актуален и богат каталог. Критерии за приемане: формулярът за добавяне на упражнение включва наименование, описание, категория и избор на мускулни групи; новото упражнение незабавно е достъпно в каталога; администраторът може да редактира и изтрива съществуващи упражнения.

ПИ-11: Като потребител, искам да мога да персонализирам своя профил с аватар и биография, за да имам идентичност в рамките на общността. Критерии за приемане: страницата на профила позволява качване на снимка; потребителят може да добавя кратка биография; промените се запазват и отразяват незабавно.

ПИ-12: Като потребител на мобилно устройство, искам интерфейсът на системата да е напълно функционален на моя смартфон, за да мога да логвам тренировките си директно от залата. Критерии за приемане: всички основни функционалности са достъпни на мобилни устройства; навигационното меню е адаптирано за малки екрани с hamburger меню; формулярите са удобни за въвеждане на данни на тъчскрийн.

3.4. Нефункционални изисквания

НФИ-01 (Производителност): Системата трябва да зарежда основните страници в рамките на 3 секунди при стандартни мрежови условия. Заявките към базата данни трябва да се изпълняват в рамките на 500 милисекунди. Системата трябва да поддържа едновременна работа на минимум 100 потребители без деградация на производителността.

НФИ-02 (Сигурност): Паролите трябва да се съхраняват в хеширан вид с използване на bcrypt алгоритъм. Системата трябва да е защитена срещу CSRF атаки чрез антифалшификационни токени. Трябва да е приложена защита срещу SQL инжекции чрез параметризирани заявки. Достъпът до административните функции трябва да е ограничен изключително за потребители с роля Admin. Потребителите не трябва да имат достъп до ресурси на други потребители.

НФИ-03 (Надеждност): Системата трябва да има uptime от минимум 99% при работа в производствена среда. Трябва да е налична стратегия за архивиране на данните. При системни грешки потребителите трябва да получават информативни съобщения без разкриване на техническа информация. НФИ-04 (Поддържаемост): Кодът трябва да следва принципите SOLID и DRY. Трябва да е налична минимум 70% покритие с

автоматизирани тестове за критичните компоненти. Кодът трябва да е документиран с XML коментари за публичните интерфейси.

3.5. Ограничения на изискванията

Следните ограничения са приети при формулирането на изискванията: системата трябва да работи с ASP.NET Core 8 и Microsoft SQL Server поради академичните изисквания на дипломния проект; потребителският интерфейс трябва да е изграден с Razor Views и Bootstrap 5 без използване на JavaScript SPA фреймворк; автентикацията трябва да използва ASP.NET Core Identity без интеграция с OAuth провайдери. Тези технологични ограничения са определени предварително и не са обект на избор в рамките на проекта.

Глава 4. Проектиране на архитектурата

4.1. Избор на архитектурен модел

Архитектурата на системата MaxFitness е базирана на класическия шаблон Model-View-Controller (MVC), реализиран чрез фреймуърка ASP.NET Core 8 MVC. MVC е доказан архитектурен шаблон за уеб приложения, осигуряващ ясно разделение на отговорностите между трите основни компонента: Модел (данни и бизнес логика), Изглед (потребителски интерфейс) и Контролер (обработка на заявките и координация). Това разделение улеснява поддържамостта, тестването и разширяемостта на системата.

В допълнение към основния MVC шаблон, в архитектурата на MaxFitness са приложени следните допълнителни архитектурни решения: Service Layer за изолиране на бизнес логиката от контролерите; Repository Pattern за абстракция на достъпа до данните; Dependency Injection за управление на зависимостите между компонентите; и DTO (Data Transfer Objects) за трансфер на данни между слоевете. Тези решения повишават модулността и тестируемостта на кода.

4.2. Компоненти на системата

Системата MaxFitness е организирана в следните основни компоненти: Presentation Layer (Razor Views + Bootstrap 5), Application Layer (Controllers + Service Layer), Domain Layer (Entities + Business Rules), Infrastructure Layer (EF Core + SQL Server), и Cross-Cutting Concerns (Authentication, Authorization, Logging, Error Handling). Тази слоеста архитектура осигурява ясни граници между компонентите и улеснява замяната на конкретни имплементации.

Системата разполага с пет основни контролера: HomeController — управлява началната страница, класацията и общата навигация; WorkoutController — управлява тренировъчния дневник (CRUD операции); ExerciseController — управлява каталога с упражнения и SVG картата на тялото; ChallengeController — управлява предизвикателствата и записването за тях; AdminController — управлява административните функции с ограничен достъп. Всеки контролер е отговорен за точно определена функционална област, спазвайки принципа на единствената отговорност.

4.3. Модели (Entities)

Доменният модел на системата включва следните основни обекти: `ApplicationUser` — разширява `IdentityUser` с допълнителни профилни полета (биография, снимка, дата на регистрация); `Workout` — представлява тренировъчна сесия с дата, продължителност, тип и бележки; `WorkoutExercise` — представлява упражнение в рамките на тренировка с серии, повторения и тегло; `Exercise` — представлява упражнение от каталога с описание, категория и асоциирани мускулни групи; `MuscleGroup` — представлява мускулна група; `Challenge` — представлява предизвикателство с цел, период и тип; `ChallengeParticipant` — представлява участието на потребител в предизвикателство.

Връзките между обектите са следните: `ApplicationUser` има много `Workouts` (1:N); `Workout` има много `WorkoutExercises` (1:N); `Exercise` е асоциирано с много `MuscleGroups` (M:N, чрез свързваща таблица); `ApplicationUser` участва в много `Challenges` чрез `ChallengeParticipant` (M:N); `Challenge` има много `ChallengeParticipants` (1:N). Тези връзки са реализирани чрез навигационни свойства в Entity Framework Core.

4.4. Контролери и маршрутизация

Маршрутизацията в системата използва конвенционалния шаблон на ASP.NET Core MVC: `{controller}/{action}/{id?}`. Допълнително са дефинирани именувани маршрути за ключови страници. Контролерите използват атрибути `[Authorize]` за защита на действия, изискващи автентикация, и `[Authorize(Roles = "Admin")]` за административни функции. Антифалшификационните токени са автоматично включени за всички POST заявки чрез `[ValidateAntiForgeryToken]`.

Моделите за изглед (`ViewModels`) са използвани за предаване на данни към изгледите, осигурявайки типова безопасност и избягвайки директното излагане на доменните модели в потребителския интерфейс. За всяко основно действие е дефиниран съответен `ViewModel` с `Data Annotations` за валидация. Частичните изгледи (`Partial Views`) са използвани за преизползваеми UI компоненти като навигацията, картите с тренировки и формулярите за упражнения.

4.5. CI/CD архитектура

Системата за непрекъсната интеграция и доставка (CI/CD) е реализирана с GitHub Actions. Тръбопроводът включва следните етапи: `Build` — компилиране на проекта с `dotnet build`; `Test` — изпълнение на всички автоматизирани тестове с `dotnet test` и генериране на доклад за покритие; `Publish` — публикуване на приложението с `dotnet`

publish; Docker Build — изграждане на Docker образ; и Deploy — деплойване на образа в производствената среда. Тръбопроводът се задейства автоматично при всеки push към клона main.

Глава 5. Избор на технологии

5.1. Сървърна рамка — ASP.NET Core 8 MVC

ASP.NET Core 8 е избран като сървърна рамка въз основа на следните критерии: зрялост и стабилност на платформата, богатата екосистема от библиотеки, вградена поддръжка за dependency injection, мощна система за middleware, и изключителна производителност (ASP.NET Core последователно заема водещи позиции в бенчмарковете на TechEmpower). Версия 8 е настоящата LTS (Long-Term Support) версия, осигуряваща поддръжка до ноември 2026 г.

В сравнение с алтернативите: Node.js/Express предлага по-малко структуриран подход и изисква повече решения от разработчика относно архитектурата; Django (Python) предлага подобно ниво на структура, но екосистемата за типово-безопасно програмиране е по-слабо развита; Laravel (PHP) е зрял и мощен фреймуърк, но C# и .NET предлагат по-добра производителност и типова безопасност. ASP.NET Core MVC предоставя оптималния баланс между структура, производителност и разработческа продуктивност за целите на проекта.

5.2. База данни — Microsoft SQL Server

Microsoft SQL Server е избран като система за управление на релационна база данни. SQL Server предлага отлична интеграция с .NET екосистемата, зрял оптимизатор на заявки, богати функции за транзакционна обработка и сигурност, и безпроблемна интеграция с Entity Framework Core. За разработката е използвана безплатната версия SQL Server Express, а за Docker деплойването — официалният образ на Microsoft SQL Server 2022.

Сравнение с алтернативите: PostgreSQL е силна алтернатива с отворен код и богата функционалност; MySQL е широко разпространен, но с по-ограничена поддръжка на сложни заявки; SQLite е подходящ за прости сценарии, но не е оптимален за многопотребителски уеб приложения. SQL Server е избран предимно поради безпроблемната интеграция с ASP.NET Core Identity и Entity Framework Core и академичния контекст на проекта.

5.3. ORM — Entity Framework Core

Entity Framework Core (EF Core) е избран като ORM (Object-Relational Mapper) за абстракция на достъпа до базата данни. EF Core е използван с подхода Code First — схемата на базата данни се генерира автоматично от C# класовете на доменния модел чрез механизма на миграциите. Това осигурява версионизирано управление на схемата, лесно синхронизиране между разработката и производствената среда, и пълен контрол върху структурата на базата данни от кода.

LINQ (Language Integrated Query) заявките, предоставяни от EF Core, позволяват типово-безопасно съставяне на заявки към базата данни директно в C# код. Функционалностите eager loading (Include), explicit loading и проекциите (Select) са използвани стратегически за оптимизация на заявките и избягване на проблема N+1. Индексирането е конфигурирано чрез Fluent API за оптимизация на честите заявки.

5.4. Изгледи — Razor Views

Razor е шаблонният език за изгледи в ASP.NET Core MVC, позволяващ смесването на C# код с HTML. За потребителския интерфейс е избран традиционният подход с Razor Views (Server-Side Rendering) пред SPA фреймуърк като React или Angular. Основанието за този избор е по-простата архитектура (без отделен API слой), по-добрата SEO оптимизация, и по-малкото количество JavaScript код за поддържане. Bootstrap 5 е използван като CSS фреймуърк за адаптивен дизайн.

5.5. Автентикация — ASP.NET Core Identity

ASP.NET Core Identity е използван за управление на потребителите, паролите, ролите и автентикацията. Identity предоставя вградени механизми за хеширане на пароли с bcrypt, управление на ролева система, cookie-базирана автентикация с защитени, криптирани cookies, и разширяем модел, позволяващ добавяне на персонализирани полета към потребителския профил. Ролевата система е конфигурирана с две роли: User (по подразбиране при регистрация) и Admin.

5.6. CI/CD — GitHub Actions

GitHub Actions е избран за реализация на CI/CD тръбопровода поради безпроблемната интеграция с GitHub (хостинг на изходния код), богатата библиотека от готови действия (actions), и безплатния tier за публични репозитории. Алтернативите Azure DevOps Pipelines и Jenkins са разгледани, но GitHub Actions е предпочетен поради

опростеността на конфигурацията и близостта до инструмента за управление на изходния код.

Тръбопроводът е дефиниран в YAML файл (.github/workflows/ci-cd.yml) и включва задействане при push към main, паралелно изпълнение на build и test стъпките, кеширане на NuGet пакетите за ускоряване на изграждането, изграждане на Docker образ с multi-stage build за минимален размер на образа, и публикуване на образа в Docker Hub. Времето за изпълнение на пълния тръбопровод е средно около 3-4 минути.

5.7. Контейнеризация — Docker

Docker е използван за контейнеризация на приложението, осигурявайки консистентна среда за изпълнение независимо от хост платформата. Dockerfile използва multi-stage build стратегия: в първия етап (build stage) приложението се компилира с образа mcr.microsoft.com/dotnet/sdk:8.0, а в крайния образ (runtime stage) се копира само компилираното приложение с по-малкия образ mcr.microsoft.com/dotnet/aspnet:8.0. Docker Compose е дефиниран за локална разработка с услуги за приложението и SQL Server.

Сравнение на технологичния стек с алтернативите: Node.js + Express + MongoDB би предложил по-бърза начална разработка, но с по-слаба типова безопасност и по-сложно управление на сложна бизнес логика. Django + PostgreSQL е силна алтернатива с богата екосистема, но Python е по-бавен от .NET при CPU-интензивни операции. Laravel + MySQL е утвърдена комбинация, но C# и .NET предлагат по-добра производителност и корпоративна поддръжка. ASP.NET Core 8 + SQL Server е оптималният избор за целите на проекта, съчетавайки производителност, типова безопасност и богата екосистема.

Глава 6. Проектиране на базата данни

6.1. Концептуален модел

Концептуалният модел на базата данни е разработен въз основа на доменния анализ и функционалните изисквания, описани в предходните глави. Моделът е представен чрез ER диаграма (Entity-Relationship Diagram), идентифицираща основните обекти, техните атрибути и взаимоотношения. Проектирането следва принципите на нормализацията до трета нормална форма (3NF) за избягване на излишъци и осигуряване на целостта на данните.

Идентифицирани са следните основни обекти: `ApplicationUsers` (потребители), `Workouts` (тренировки), `WorkoutExercises` (упражнения в тренировка), `Exercises` (каталог с упражнения), `MuscleGroups` (мускулни групи), `ExerciseMuscleGroups` (свързваща таблица), `Challenges` (предизвикателства) и `ChallengeParticipants` (участници в предизвикателство). Таблиците на ASP.NET Core Identity (`AspNetUsers`, `AspNetRoles`, `AspNetUserRoles` и др.) са включени автоматично при конфигурирането на Identity с EF Core.

6.2. Описание на таблиците

Таблица `ApplicationUsers` разширява стандартната таблица `AspNetUsers` на Identity с допълнителни колони: `Bio` (`nvarchar(500)`, nullable) за биография на потребителя; `ProfilePictureUrl` (`nvarchar(500)`, nullable) за URL на профилната снимка; `CreatedAt` (`datetime2`, not null) за дата на регистрация. Връзката с таблицата `AspNetUsers` е реализирана чрез наследяване в EF Core — `ApplicationUser` наследява `IdentityUser` и допълнителните полета се добавят в същата таблица `AspNetUsers`.

Таблица `Workouts` съдържа следните колони: `Id` (int, PK, IDENTITY), `UserId` (`nvarchar(450)`, FK към `AspNetUsers`), `Date` (`datetime2`, not null), `DurationMinutes` (int, not null), `WorkoutType` (`nvarchar(50)`, not null), `Notes` (`nvarchar(1000)`, nullable), `CreatedAt` (`datetime2`, not null). Индекс е дефиниран върху колоната `UserId` за ускоряване на заявките за тренировки на конкретен потребител. Каскадно изтриване е конфигурирано — при изтриване на потребител, всички негови тренировки се изтриват.

Таблица `WorkoutExercises` съдържа: `Id` (int, PK, IDENTITY), `WorkoutId` (int, FK към `Workouts`), `ExerciseId` (int, FK към `Exercises`), `Sets` (int, not null), `Reps` (int, not null), `WeightKg` (`decimal(6,2)`, nullable), `Notes` (`nvarchar(500)`, nullable). Таблицата реализира

релацията M:N между тренировки и упражнения с допълнителни атрибути. Индекс е дефиниран върху WorkoutId за ускоряване на зареждането на упражненията на дадена тренировка.

Таблица Exercises съдържа: Id (int, PK, IDENTITY), Name (nvarchar(100), not null, UNIQUE), Description (nvarchar(2000), nullable), Category (nvarchar(50), not null), DifficultyLevel (nvarchar(20), not null), ImageUrl (nvarchar(500), nullable). Уникален индекс върху Name гарантира, че не могат да съществуват две упражнения с едно и също наименование. Таблицата се поддържа от администраторите.

Таблица MuscleGroups съдържа: Id (int, PK, IDENTITY), Name (nvarchar(50), not null, UNIQUE), SvgId (nvarchar(50), not null) — идентификаторът на съответния елемент в SVG картата на тялото. Таблица ExerciseMuscleGroups е свързваща за реализиране на M:N релацията между упражнения и мускулни групи: ExerciseId (int, FK) и MuscleGroupId (int, FK), като двете колони заедно образуват съставен първичен ключ.

Таблица Challenges съдържа: Id (int, PK, IDENTITY), Title (nvarchar(100), not null), Description (nvarchar(1000), nullable), ChallengeType (nvarchar(50), not null — WorkoutCount, TotalDuration, ExerciseCount), TargetValue (int, not null), StartDate (datetime2, not null), EndDate (datetime2, not null), IsActive (bit, not null). Таблица ChallengeParticipants съдържа: Id (int, PK, IDENTITY), ChallengeId (int, FK), UserId (nvarchar(450), FK), JoinedAt (datetime2, not null), CurrentProgress (int, not null, default 0), IsCompleted (bit, not null, default false). Уникален индекс върху (ChallengeId, UserId) предотвратява дублиращи записвания.

6.3. Миграции на Entity Framework Core

Схемата на базата данни се управлява изцяло чрез механизма на миграциите на EF Core. Всяка промяна в доменния модел генерира нова миграция, описваща трансформацията на схемата. Миграциите се прилагат автоматично при стартиране на приложението в режим на разработка чрез извикването на context.Database.MigrateAsync() в Program.cs. За производствената среда миграциите се прилагат като стъпка от CI/CD тръбопровода преди деплойването на новата версия.

6.4. Индекси и оптимизация

Индексирането е конфигурирано стратегически за оптимизация на честите заявки. Основните индекси включват: некластеризиран индекс върху Workouts.UserId за бързо

извличане на тренировките на потребител; некластеризиран индекс върху Workouts.Date за хронологично сортиране; некластеризиран индекс върху WorkoutExercises.WorkoutId за бързо зареждане на упражненията на тренировка; некластеризиран индекс върху ChallengeParticipants.UserId за бързо намиране на предизвикателствата на потребител; и некластеризиран индекс върху Challenges.IsActive за филтриране на активни предизвикателства.

Глава 7. Реализация на сървърната част

7.1. Структура на проекта

Проектът е организиран в следната директорна структура: `Controllers/` — съдържа петте основни контролера; `Models/` — съдържа доменните модели (entities) и `ViewModels`; `Views/` — съдържа Razor изгледите, организирани в поддиректории по контролер; `Services/` — съдържа интерфейсите и имплементациите на сервизния слой; `Data/` — съдържа `ApplicationDbContext` и миграциите; `wwwroot/` — съдържа статичните ресурси (CSS, JavaScript, изображения). Конфигурационните файлове (`Program.cs`, `appsettings.json`) са в корена на проекта.

7.2. Контролери

`WorkoutController` реализира пълен CRUD за тренировъчните сесии. Методите `Index` (GET) и `Index` (POST за филтриране) зареждат пагинирания дневник на потребителя. `Create` (GET) зарежда формуляра за нова тренировка с каталога от упражнения; `Create` (POST) валидира модела и записва тренировката. `Edit` (GET) зарежда формуляра за редактиране; `Edit` (POST) прилага промените. `Delete` (POST) изтрива тренировката след проверка на собствеността. `Details` (GET) показва детайлна информация за тренировката.

`ChallengeController` управлява предизвикателствата. `Index` (GET) зарежда всички активни предизвикателства с информация за записването на текущия потребител и прогреса му. `Join` (POST) записва потребителя за предизвикателство след проверка за дублиращо записване. `Leave` (POST) отписва потребителя. `AdminController` е защитен с `[Authorize(Roles = "Admin")]` на ниво клас. `Dashboard` (GET) зарежда обобщена статистика. Методите за управление на потребители, упражнения и предизвикателства са имплементирани с пълна CRUD функционалност.

7.3. Сервизен слой

Сервизният слой изолира бизнес логиката от контролерите, осигурявайки по-лесно тестване и повторно използване. Основните сервиз-интерфейси са: `IWorkoutService` с методи за CRUD операции и статистики; `IExerciseService` за управление на каталога; `IChallengeService` за управление на предизвикателствата и прогреса; `ILeaderboardService` за изчисляване на класациите; и `IStatsService` за агрегирана

аналитика. Всяка имплементация на интерфейс е регистрирана в DI контейнера с `Scoped lifetime`.

Логиката за актуализиране на прогреса в предизвикателствата е особено интересна от архитектурна гледна точка. При добавяне или изтриване на тренировка, `WorkoutService` извиква `IChallengeService.UpdateProgressAsync(userId)` за актуализиране на прогреса на потребителя в активните предизвикателства. Прогресът се изчислява в зависимост от типа предизвикателство: за `WorkoutCount` — брой тренировки в периода; за `TotalDuration` — сума на продължителностите; за `ExerciseCount` — брой уникални упражнения.

7.4. Repository Pattern и достъп до данни

В системата е приложен опростен вариант на `Repository Pattern`, при който сервизите използват `ApplicationDbContext` директно чрез `Dependency Injection`, без допълнителен слой от репозитории. Това решение е прието съзнателно — добавянето на пълноценен репозиторен слой над `EF Core` в много случаи е излишна абстракция, тъй като `DbContext` вече осигурява `unit of work` и колекционна абстракция. При нужда от мокване в тестовете, `ApplicationDbContext` е заменен с `InMemory` провайдър на `EF Core`.

7.5. Валидация

Валидацията се извършва на два нива: клиентска валидация чрез `jQuery Unobtrusive Validation` (автоматично генерирана от `Data Annotations`) за незабавна обратна връзка; и сървърна валидация чрез `ModelState` в контролерите за гарантирана сигурност независимо от клиента. `Data Annotations` (`[Required]`, `[StringLength]`, `[Range]`, `[EmailAddress]` и др.) са дефинирани върху `ViewModel` класовете. Персонализирани валидационни атрибути са реализирани за специфични бизнес правила.

7.6. Сигурност

Сигурността е реализирана на множество нива. Автентикацията е обезпечена от `ASP.NET Core Identity` с `cookie`-базирани сесии. Авторизацията използва атрибути `[Authorize]` и `[Authorize(Roles = "Admin")]` за защита на ресурсите. `CSRF` защитата е автоматична за всички `POST` форми чрез `AntiforgeryToken`. `SQL` инжекциите са предотвратени чрез параметризираните заявки на `EF Core`. Достъпът до ресурси на други потребители е проверяван в контролерите — например, при опит за редактиране

на чужда тренировка, потребителят получава 403 Forbidden. Всички съобщения за грешки са потребителски, без технически детайли.

Глава 8. Реализация на клиентската част

8.1. Обща концепция на потребителския интерфейс

Потребителският интерфейс на MaxFitness е проектиран с фокус върху лекота на употреба, визуална яснота и мотивиращ дизайн. Основните принципи са: минималистичен дизайн без излишни елементи; консистентна цветова схема с тъмни нюанси и акцентни цветове в зелено и синьо, подходящи за фитнес тематиката; интуитивна навигация с ясна йерархия; и адаптивен дизайн, функциониращ безупречно на устройства от 320px до 2560px ширина. Bootstrap 5 е използван като CSS фреймуърк, допълнен с персонализирани CSS правила в отделен stylesheet.

Навигацията е реализирана с адаптивна лента за навигация (navbar), скриваща се в hamburger меню на мобилни устройства. В десктоп режим навигацията показва всички основни раздели: Начало, Тренировки, Упражнения, Предизвикателства, Класация и Статистики. Вдясно се показват аватарът на потребителя и бутонът за изход. Навигацията е частичен изглед (_Navbar.cshtml), включван в главния layout (_Layout.cshtml), и използва asp-controller и asp-action tag helpers за генериране на правилните URL адреси.

8.2. Начална страница

Началната страница (HomeController.Index) е различна за автентикирани и неавтентикирани потребители. За посетители се показва маркетинговата страница с описание на функционалностите, скрийншоти и бутони за регистрация и вход. За автентикирани потребители страницата показва персонализирано табло: броя тренировки за текущата седмица, последните 5 тренировки с бързи линкове, активните предизвикателства с прогрес-барове, и позицията в класацията. Тази персонализация е реализирана чрез проверка на User.Identity.IsAuthenticated в изгледа и предаване на ViewModel с персонализирани данни от контролера.

8.3. Тренировъчен дневник

Страницата на тренировъчния дневник (WorkoutController.Index) показва хронологичен списък с тренировките на потребителя, пагиниран по 10 на страница. Всяка тренировъчна карта показва датата, типа тренировка, продължителността, броя упражнения и кратко резюме. Бутоните за редактиране и изтриване са достъпни директно от картата. Над списъка е поставен бутон за добавяне на нова тренировка.

Формулярът за нова тренировка (`WorkoutController.Create`) използва динамично добавяне на упражнения чрез JavaScript. Потребителят избира упражнение от падащо меню (`Select2` компонент за по-добро UX), след което попълва серии, повторения и тегло. Бутонът „Добави упражнение“ добавя нов ред динамично без презареждане на страницата. При подаване на формуляра, всички упражнения се изпращат към сървъра. Валидацията е интегрирана за всички задължителни полета.

8.4. Каталог с упражнения и SVG карта на тялото

Страницата с каталога от упражнения (`ExerciseController.Index`) предлага два режима на разглеждане: списъчен вид с всички упражнения и филтриране по категория и мускулна група; и интерактивен SVG вид с карта на тялото. Интерактивната SVG карта на тялото е централна функционалност на системата — тя визуализира предна и задна проекция на човешкото тяло с всички основни мускулни групи, оцветени интерактивно. При кликване върху мускулна група, страницата се актуализира (без пълно презареждане чрез AJAX) и показва упражненията, работещи тази мускулна група.

SVG картата е реализирана с вградени path елементи за всяка мускулна група, с `data`-атрибути (`data-muscle-id`) свързващи SVG елементите към записите в базата данни. JavaScript кодът обработва `hover` и `click` събитията, осветявайки засегнатите мускули с accent цвят. При `hover` се показва tooltip с името на мускулната група. Тази функционалност е реализирана изцяло с `vanilla JavaScript` без допълнителни библиотеки.

8.5. Предизвикателства и класация

Страницата с предизвикателства (`ChallengeController.Index`) показва карти за всяко активно предизвикателство. Всяка карта съдържа заглавие, описание, тип, цел, срок, и брой участници. За потребители, записани за предизвикателството, се показва прогрес-бар с текущия напредък към целта. Бутонът „Присъедини се“ или „Напусни“ се показва контекстуално. Завършените предизвикателства са маркирани с визуален индикатор за успех.

Страницата с класацията (`HomeController.Leaderboard`) показва топ потребители по брой тренировки за текущата седмица и месец. Таблото с класация включва подредба с медали (злато, сребро, бронз) за топ 3 позиции. Текущият потребител е визуално

открит в класацията. При позиция извън топ 10, в дъното се показва записът на текущия потребител с неговата позиция, за да знае колко далеч е от топ 10.

8.6. Профил, статистики и администрация

Страницата на потребителския профил показва аватара, биографията, датата на регистрация и обобщена статистика. Формулярът за редактиране на профила позволява промяна на биографията и качване на нова профилна снимка. Страницата на статистиките (`WorkoutController.Stats`) визуализира: общ брой тренировки, общо изразходвани минути, средна продължителност на тренировка, хит-мап на активността по дни от седмицата, графика на тренировките по месеци за последните 6 месеца, и топ 5 упражнения по брой изпълнения. Графиките са реализирани с библиотеката `Chart.js`.

Административният панел (`AdminController.Dashboard`) е достъпен само за потребители с роля `Admin`. Той предоставя обобщена информация: общ брой потребители, общ брой тренировки, брой активни предизвикателства, и последните регистрирани потребители. Навигацията в административния панел включва раздели за управление на потребители (с права за деактивиране и задаване на роля `Admin`), управление на упражнения (пълна CRUD) и управление на предизвикателства (пълна CRUD). Административните страници са стилизирани с предупредителни визуални индикатори за ясно разграничение от обичайния потребителски интерфейс.

8.7. Адаптивен дизайн

Адаптивният дизайн е реализиран чрез `Bootstrap 5 grid` системата и медийни заявки. Основните breakpoints са: `xs` ($< 576\text{px}$) за малки смартфони, `sm` ($576\text{-}767\text{px}$) за смартфони, `md` ($768\text{-}991\text{px}$) за планшети, `lg` ($992\text{-}1199\text{px}$) за лаптопи, `xl` ($\geq 1200\text{px}$) за десктоп. На мобилни устройства навигацията се трансформира в `hamburger` меню; тренировъчните карти заемат пълната ширина; таблиците се адаптират с хоризонтален `scroll` или стакиране на колоните; и `SVG` картата на тялото се мащабира пропорционално.

Глава 9. Тестване

9.1. Стратегия за тестване

Стратегията за тестване на системата MaxFitness следва пирамидалния модел на тестване: основата е изградена от единични тестове (unit tests) за бизнес логиката и сервисния слой; в средата са интеграционните тестове за проверка на взаимодействието между компонентите и базата данни; в горния слой са функционалните тестове за проверка на потребителските сценарии. Автоматизираните тестове са написани с xUnit като тестови фреймуърк, Moq за мокване на зависимости, и EF Core InMemory провайдър за тестване на компоненти, зависими от базата данни.

9.2. Единични тестове

Единичните тестове покриват сервисния слой на приложението. Общо са написани 47 преминаващи единични теста, организирани в следните тестови класове: WorkoutServiceTests (16 теста) — тества CRUD операциите, пагинацията, филтрирането и статистиките; ChallengeServiceTests (12 теста) — тества записването, напусването и актуализирането на прогреса; ExerciseServiceTests (8 теста) — тества CRUD операциите за упражнения и търсенето; LeaderboardServiceTests (6 теста) — тества изчисляването на класациите; и StatsServiceTests (5 теста) — тества агрегираните статистики.

Примерен тест от WorkoutServiceTests демонстрира подхода: методът CreateWorkoutAsync_WithValidData_ShouldPersistWorkout проверява, че при подаване на валидни данни за нова тренировка, тя се записва в базата данни с правилните стойности. Тестът използва InMemory база данни, извиква CreateWorkoutAsync и верифицира, че записът съществува в базата с очакваните стойности. Тестовите следват шаблона Arrange-Act-Assert за яснота и поддръжаемост.

9.3. Интеграционни тестове

Интеграционните тестове проверяват взаимодействието между контролерите и сервисния слой, използвайки WebApplicationFactory за стартиране на тестово копие на приложението. Тестовите обхващат: успешното зареждане на главните страници с правилен HTTP статус код; редиректирането на неавтентикирани потребители към страницата за вход при опит за достъп до защитени ресурси; коректната обработка на

формуляри с валидни и невалидни данни; и проверка на ролевата авторизация за административните ресурси.

9.4. Браузърно тестване

Браузърното тестване е проведено мануално на следните браузъри и устройства: Google Chrome 120 (десктоп и мобилен), Mozilla Firefox 121 (десктоп), Microsoft Edge 120 (десктоп), Safari 17 (iOS, iPhone 14), и Chrome on Android (Samsung Galaxy S22). Всички основни потребителски сценарии са тествани и верифицирани на всяка платформа. Адаптивният дизайн е верифициран на разделителни способности от 320px до 2560px.

9.5. Оптимизация и резултати

В процеса на тестване са идентифицирани и отстранени множество проблеми с производителността. Проблемът N+1 при зареждането на дневника с тренировки е решен чрез добавяне на `Include(w => w.WorkoutExercises).ThenInclude(we => we.Exercise)` в заявката. Индексирането е оптимизирано след анализ с SQL Server Execution Plan. Кеширането на Response е внедрено за статичните ресурси с дълъг `cache-control` header. Времето за зареждане на главните страници е измерено с Lighthouse и се движи в рамките на 1-2 секунди.

Резултатите от изпълнението на тестовете: 47 единични теста — 47 преминаващи, 0 неуспешни; интеграционни тестове — 18 преминаващи; общо покритие на кода с тестове — 74% за сервизния слой. CI/CD тръбопроводът изпълнява всички тестове автоматично при всеки push към main клона, осигурявайки непрекъсната верификация на функционалността.

Глава 10. Деплойване и заключение

10.1. CI/CD тръбопровод с GitHub Actions

CI/CD тръбопроводът е дефиниран в `.github/workflows/ci-cd.yml` и се задейства автоматично при push към клона `main`. Тръбопроводът изпълнява следните стъпки в указания ред: Checkout на изходния код; Настройка на .NET 8 SDK среда; Възстановяване на NuGet зависимостите с `dotnet restore`; Компилиране на проекта с `dotnet build --no-restore --configuration Release`; Изпълнение на тестовете с `dotnet test --no-build --verbosity normal`; Публикуване на приложението с `dotnet publish`; Изграждане на Docker образ с тагове `latest` и с номера на `commit`; и публикуване на образа в Docker Hub.

10.2. Docker контейнеризация

Dockerfile използва `multi-stage build` за минимизиране на размера на крайния образ. В `build stage` се използва образът `mcr.microsoft.com/dotnet/sdk:8.0-alpine` за компилиране, а в `runtime stage` — `mcr.microsoft.com/dotnet/aspnet:8.0-alpine`. Alpine базираните образи осигуряват значително по-малки образи (около 120 MB спрямо около 300 MB за `debian-based`). Непривилегирован потребител (`appuser`) е конфигуриран за изпълнение на процеса за повишена сигурност. Health check е конфигуриран за мониторинг на здравето на контейнера.

Docker Compose файлът дефинира два сервиза: `web` (приложението `MaxFitness`) и `db` (Microsoft SQL Server 2022). Сервизите са свързани в обща мрежа. Volumes са конфигурирани за персистиране на данните на SQL Server между рестартирания на контейнера. Environment variables за `connection string` и JWT секрети се предават чрез `.env` файл, който не е включен в хранилището на изходния код.

10.3. Производствена конфигурация

Производствената конфигурация се управлява чрез `appsettings.Production.json` и `environment variables`. Чувствителните данни (`connection string`, секрети) се предават изключително чрез `environment variables` и не се съхраняват в хранилището. HTTPS е конфигуриран чрез SSL/TLS сертификат. HTTP Strict Transport Security (HSTS) е активиран. Логирането в производствена среда е конфигурирано с ниво `Warning` за намаляване на обема на логовете. Application Insights е интегрирана за мониторинг на производителността и грешките.

10.4. Резултати от разработката

Системата MaxFitness е разработена успешно в рамките на дипломния проект, реализирайки всички 24 функционални изисквания и 12 потребителски истории. Изградена е пълнофункционална уеб система с 5 контролера, 15+ Razor изгледа, 7 сервиза, 8 доменни модела и 47 преминаващи автоматизирани теста. CI/CD тръбопроводът осигурява автоматизирана верификация при всяка промяна в кода. Системата е контейнеризирана с Docker и готова за деплойване в производствена среда.

10.5. Научни и практически приноси

Настоящият дипломен проект демонстрира успешното проектиране и разработване на многослойна уеб система с ASP.NET Core 8 MVC, следваща принципите на чистата архитектура и добрите практики в разработката на софтуер. Практическите приноси включват: реализация на система за проследяване на фитнес тренировки с интерактивна SVG визуализация на мускулните групи; имплементация на gamification елементи (предизвикателства, класация) за повишаване на мотивацията на потребителите; и демонстрация на пълен CI/CD тръбопровод с GitHub Actions и Docker за образователни цели.

10.6. Насоки за бъдещо развитие

Бъдещото развитие на системата MaxFitness може да включва следните направления: разработване на мобилно приложение (iOS и Android) с React Native или .NET MAUI за по-добро мобилно изживяване; интеграция с API на носими устройства (Garmin, Fitbit, Apple Health) за автоматично импортиране на тренировки; добавяне на хранителен дневник с база данни от храни за цялостно здравно проследяване; реализация на AI-базирани препоръки за тренировъчни програми въз основа на историята и целите на потребителя; и въвеждане на социални функции — добавяне на приятели, споделяне на тренировки, групови предизвикателства.

От техническа гледна точка системата може да бъде подобрена чрез: миграция към микросервизна архитектура при мащабиране; внедряване на Redis за кеширане на честите заявки; интеграция на пълнотекстово търсене с Elasticsearch за каталога с упражнения; и добавяне на real-time функционалности с SignalR за актуализиране на класациите в реално време.

10.7. Заключение

Настоящият дипломен проект постига поставените цели — разработена е пълнофункционална, сигурна и производителна система за управление на фитнес тренировки. Системата MaxFitness демонстрира успешното приложение на модерен технологичен стек (ASP.NET Core 8, SQL Server, EF Core, Bootstrap 5, Docker, GitHub Actions) в реален образователен проект. Архитектурата следва принципите на разделение на отговорностите, изолацията на бизнес логиката и тестируемостта на кода.

В процеса на разработката са придобити ценни практически умения в областите на уеб разработката с ASP.NET Core, проектирането на релационни бази данни, тестването на софтуер, и DevOps практиките с Docker и GitHub Actions. Проектът потвърждава, че задълбоченото познаване на архитектурните принципи и инструментите на разработката позволява реализирането на качествен, поддържаем и разширяем софтуер в рамките на ограниченото време на дипломен проект.

Системата MaxFitness е реализирана като отворена за разширение платформа, която може да бъде развита в пълноценен продукт при добавяне на допълнителни ресурси. Изградената архитектурна основа, богатата тестова покритост и автоматизираният CI/CD тръбопровод осигуряват стабилна база за бъдещо развитие. Проектът демонстрира, че съвременните технологии и добрите инженерни практики позволяват на малък екип да разработи конкурентоспособна система в разумен срок.